

Recursive Functions

CSCI 1101

2024-04-16

Recursive Functions

In order to compute its result a function may call other functions.

One of those functions may be *itself*.

A function that calls itself in the process of computing its result is called a **recursive function**.

```
def countdown(n) :      # sig: (int) --> NoneType
    if n <= 0 :
        print('go!')      # base case
    else :
        print(f'{n}')
        countdown(n-1)    # recursive call
```

There is a **case distinction** to decide whether the function needs to call itself.

A branch where the function doesn't call itself is a **base case**.

A self-call of the function is a **recursive call**.

Example: Countdown

Many programming problems can be solved by using either iteration or recursion:

```
def countdown_iter(n) :  
    while n > 0 :  
        print(f'{n}')        n -= 1  
    print('go!')
```

$n \mapsto \overbrace{n, (n-1), (n-2), \dots, 2, 1}^{\text{count down from } n}, \underbrace{\text{go!}}_{\text{count down from } n-1}$

```
def countdown_rec(n) :  
    if n <= 0 :                # count down from zero.  
        print('go!')    else :                     # count down from a positive number,  
        print(f'{n}')         # using how to count down  
        countdown_rec(n-1)    # from its predecessor.
```

Iteration vs Recursion

Problem solving:

Iteration- builds up solutions using loops and accumulators or effects.

Recursion- builds up solutions using the results of recursive calls.

Making progress:

Iteration- loop variables get “closer” to failing the guard condition.

Recursion- recursive call arguments get “closer” to a base case.

Combining results:

Iteration- using programmer-supplied accumulator update function.

Recursion- using built-in function-calling mechanism of the language.

Recursive Problem-Solving

To solve a problem by recursion:

- Identify the cases for which you can solve the problem directly.
These are the *base cases*.
- For any remaining cases, work out how to use the solution to a *smaller problem* (smaller number, shorter list, etc.) to solve the current problem.
- In each case, write out how to solve the problem for *just the current case*.
Don't worry about how to solve the other cases or recursive subproblems!

This strategy will work so long as you:

- cover all possible cases,
- do the right thing in each case,
- ensure the subproblems are getting smaller.

Example: Factorial

The **factorial** function multiplies all of the positive integers up to a given number:

```
def factorial_iter(n) :
```

```
    acc = 1
```

```
    while n > 0 :
```

```
        acc *= n
```

```
        n -= 1
```

```
    return acc
```

$$n \quad \mapsto \quad \underbrace{n \times (n-1) \times (n-2) \times \dots \times 2 \times 1}_{\text{factorial of } n-1} \quad \text{factorial of } n$$

```
def factorial_rec(n) :
```

```
    if n == 0 : # compute the factorial of zero:
```

```
        return 1
```

```
    elif n > 0 : # compute the factorial of a positive number,
```

```
        pred_fact = factorial_rec(n-1) # using the factorial
```

```
        return n * pred_fact # of its predecessor
```

List Recursion

Here is a common strategy for writing recursive functions for lists.

A list (xs) is either empty or it is not.

An empty list has no smaller slices, so it must be a *base case*.

A non-empty list always contains an *element* at index 0 ($xs[0]$).

This is called the **head** element.

A non-empty list also contains a *slice* including all but the head element ($xs[1:]$).

This is called the **tail** list.

Often a *recursive call* is made on the tail list, which is always one element shorter than the original list.

Example: List Summing

Compute the sum of the numbers in a list:

```
def sum_iter(nums) :  
    acc = 0  
    for num in nums :  
        acc += num  
    return acc
```

$$[n_0, n_1, n_2, \dots, n_k] \quad \longmapsto \quad \overbrace{n_0 + n_1 + n_2 + \dots + n_k}^{\text{sum(nums)}} \\ \text{sum(nums[1:])}$$

```
def sum_rec(nums) :  
    if nums == [] :           # sum an empty list of numbers:  
        return 0  
    else :                    # sum a nonempty list of numbers,  
        tail_sum = sum_rec(nums[1: ])      # using the sum  
        return nums[0] + tail_sum          # of its tail
```


Self-Similar Shapes

A *self-similar* shape is one whose description is recursive, i.e., it contains a reference to itself.

```
def tree(size , depth) :    # sig: (int , int) --> NoneType
    if depth <= 0 :         # draw a tree of depth 0
        pass
    else :                  # draw a tree of positive depth
        turtle.forward(size)
        turtle.left(30.0)
        tree(4/5*size , depth-1) # using two trees that are
        turtle.right(60.0)
        tree(4/5*size , depth-1) # each one unit shallower
        turtle.left(30.0)
        turtle.back(size)
```

To draw faster use `turtle.speed(0)` or `turtle.tracer(300 , 100)`

Fractal Triangles

Starting with an equilateral triangle we can draw a scaled-down version of it in each of its three corners and scaled down version of those in each of their corners and so on and so on...

The result is a **Sierpinski triangle**:

```
def sierpinski_triangle(size , depth) :  
    if depth <= 0 :  
        pass  
    else :  
        for i in range(3) :  
            sierpinski_triangle(size/2 , depth-1)  
            turtle.forward(size)  
            turtle.left(120.0)
```

To Do

Finish *Homework 10: Object-Oriented Programming* on CodeRunner by 11:00 this Thursday.

Finish *Project 3 - Automated Grader (Part 2)* on CodeRunner by midnight this Friday.